LAB Manual – 1

Code/Course: 23CSE312/Distributed Systems

Topic: Distributed Applications using Sockets(Chat Program)

[CO05] [BTL-3]
Student Name: D Ganeswara
Roll No.: AM.SC.U4CSE23119

Batch/Section: CSE-B
Date: 28/01/2026

1. Aim

To build a basic distributed chat application using Python sockets with:

- **Part A: Iterative server and client** (single connection served at a time).
- **Part B: Concurrent server and client** (multiple clients served simultaneously).

Task 1) Iterative-server and client

Create a most basic distributed application consisting of a client and a server program which can chat with each other. Make the server iterative, i.e., accept and serve a single connection at a time. Follow the steps below:

a) Try the "simple server-client program" from the link given before
b) Modify it for the chat application
c) Once the above works in your system, pair up with your neighbour and try the client and server across two systems connected in the same network

```
❯ cd Amrita/distributed_systems/labsheet1
❯ python3 server1.py
Server listening on 127.0.0.1:12345
Accepted connection from ('127.0.0.1', 62008)
Received message: hey
Received message: hi
```

```
❯ cd Amrita/distributed_systems/labsheet1
❯ python3 client1.py
Enter your message: hey
Server response: Server received your message: hey
Enter your message: hi
Server response: Server received your message: hi
Enter your message:
```

From other client:

```
❯ python3 server1.py
Server listening on 172.20.10.6:12345
Accepted connection from ('172.20.10.3', 56699)
Received message: gani
Received message: hi
Received message: uwu
```

```
PS D:\OneDrive\Desktop\Amrita\S6\Distributed Lab\Lab1> python client2.py
Enter your message: gani
Server response: Server received your message: gani
Enter your message: hi
Server response: Server received your message: hi
Enter your message: uwu
Server response: Server received your message: uwu
```

Code:

Server.py:

```
import socket
import threading



def handle_client(client_socket):
while True:
data = client_socket.recv(1024)
if not data:
break
message = data.decode('utf-8')
print(f"Received message: {message}")
response = "Server received your message: " + message
client_socket.sendall(response.encode('utf-8'))
client_socket.close()



def main():
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
host = '127.0.0.1'
port = 12345
server_socket.bind((host, port))
server_socket.listen(5)
print(f"Server listening on {host}:{port}")

while True:
client_socket, client_address = server_socket.accept()
print(f"Accepted connection from {client_address}")
client_handler = threading.Thread(target=handle_client, args=(client_socket,))
client_handler.start()

if __name__ == "__main__":
main()
```

Client.py:

```
import socket
import threading

def main():
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
host = '127.0.0.1'
port = 12345
client_socket.connect((host, port))

while True:
message = input("Enter your message: ")
client_socket.sendall(message.encode('utf-8'))
data = client_socket.recv(1024)
response = data.decode('utf-8')
print(f"Server response: {response}")

if __name__ == "__main__":
main()
```

Task 2) Concurrent-server and client

Modify the chat program to make the server concurrent, i.e., the server can establish connections with multiple clients simultaneously.

Refeerence: Multi-threading

Study the associated socket concepts from the above link.

```
❯ python3 client.py
[*] Connecting to 127.0.0.1:5002...
[+] Connected.
Enter your name: gani

[2026-01-28 09:42:41] Varshenee: hi guys
hi

[2026-01-28 09:49:10] Anabha: hi

[2026-01-28 09:49:19] Varshenee: how are you

[2026-01-28 09:49:22] Varshenee: take care
hey guys hows
[2026-01-28 09:49:30] Anabha: good vibes only
 life

[2026-01-28 09:49:41] Varshenee: amazing!!!
```

```
❯ python3 server.py
[*] Listening as 0.0.0.0:5002
[+] ('172.20.10.3', 61530) connected.
[+] ('127.0.0.1', 61286) connected.
[+] ('172.20.10.4', 55824) connected.
[+] ('172.20.10.3', 49819) connected.
[!] Error: [Errno 60] Operation timed out
[!] Error: [Errno 54] Connection reset by peer
[+] ('172.20.10.3', 63631) connected.
[+] ('127.0.0.1', 61686) connected.
[!] Error: [Errno 54] Connection reset by peer
[+] ('172.20.10.4', 65514) connected.
[!] Error: [Errno 54] Connection reset by peer
[+] ('172.20.10.4', 54805) connected.
[!] Error: [Errno 54] Connection reset by peer
[+] ('172.20.10.4', 54818) connected.
```

Code:

Server.py:

```
import socket
from threading import Thread

SERVER_HOST = "0.0.0.0"
SERVER_PORT = 5002
separator_token = "<SEP>"
client_sockets = set()
s = socket.socket()
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind((SERVER_HOST, SERVER_PORT))
s.listen(5)
print(f"[*] Listening as {SERVER_HOST}:{SERVER_PORT}")
def listen_for_client(cs):
while True:
try:
msg = cs.recv(1024).decode()
if not msg:
client_sockets.remove(cs)
cs.close()
break
except Exception as e:
print(f"[!] Error: {e}")
client_sockets.remove(cs)
cs.close()
break
else:
msg = msg.replace(separator_token, ": ")
for client_socket in client_sockets:
if client_socket != cs:
client_socket.send(msg.encode())

while True:
client_socket, client_address = s.accept()
print(f"[+] {client_address} connected.")
```

```
client_sockets.add(client_socket)
t = Thread(target=listen_for_client, args=(client_socket,))
t.daemon = True
t.start()
```

Client.py:

```
import socket
import random
from threading import Thread
from datetime import datetime
SERVER_HOST = "127.0.0.1"
SERVER_PORT = 5002
separator_token = "<SEP>"
s = socket.socket()
print(f"[*] Connecting to {SERVER_HOST}:{SERVER_PORT}...")
s.connect((SERVER_HOST, SERVER_PORT))
print("[+] Connected.")

name = input("Enter your name: ")

def listen_for_messages():
while True:
message = s.recv(1024).decode()
print("\n" + message)
t = Thread(target=listen_for_messages)
t.daemon = True
t.start()
while True:
to_send = input()
if to_send.lower() == 'q':
break
date_now = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
to_send = f"[{date_now}] {name}{separator_token}{to_send}"
s.send(to_send.encode())
s.close()
```